

3.10 - Banco de dados em larga escala

O gargalo que aparece mais cedo do que se espera

Estratégias para escalar o banco

Quatro alavancas antes do sharding

- **Indexação estratégica é o primeiro e mais barato ganho**
 - Índice certo transforma varredura de tabela em busca direta
- **Connection pooling reaproveita conexões caras de abrir**
 - Evita exaurir o banco com milhares de conexões
- **Read replicas separam leitura pesada da escrita**
 - Cópias somente leitura absorvem relatórios e consultas
- **Sharding particiona os dados entre vários bancos**
 - Última alavanca, de alta complexidade operacional

Relatório que trava o tempo real

- **Um único PostgreSQL atende tracking, relatórios e config**
 - Tracking precisa gravar em menos de duzentos milissegundos

LUNALOG

O PostgreSQL da LunaLog fazia tudo ao mesmo tempo: updates de tracking em tempo real, queries pesadas de relatório histórico e leituras de configuração. As queries de relatório começaram a criar locks que atrasavam os updates de tracking, justamente os que precisavam ser gravados em menos de duzentos milissegundos para manter a promessa de tempo real. O time apontou os relatórios para read replicas, separando a carga analítica de leitura da carga operacional de escrita. O tempo real voltou a ser real, e nenhuma linha da aplicação precisou mudar.

Polyglot persistence e CAP

Escolher o banco certo para cada carga

- **Polyglot persistence usa o banco ideal por necessidade**
 - Relacional, documento, chave-valor e séries temporais juntos
- **CAP afirma que não dá para ter os três sob partição**
 - Consistência, disponibilidade e tolerância a partição
- **Sob falha de rede, escolha consistência ou disponibilidade**
 - Tracking aceita disponibilidade, billing exige consistência
- **Não existe banco universal para todos os problemas**

Seção 3: o arsenal da escala

- ✓ Statelessness habilita scale-out horizontal e elasticidade
- ✓ Cache, filas e read replicas tiram carga do caminho crítico
- ▶ Lei de Conway molda a arquitetura pela estrutura do time
- ▶ Cada protocolo, REST, gRPC ou GraphQL, resolve uma camada
- ! Sem rate limit e circuit breaker, a falha de um vira a de todos
- ! O banco é o gargalo mais comum, planeje antes do pico



Os fundamentos de escala estão no lugar: horizontal e vertical, cache, filas, rate limiting, circuit breaker e banco. A próxima aula é prática, vamos analisar contratos de API reais com problemas de propósito e refazê-los ao vivo. E logo depois, a seção 4 começa com uma pergunta que muda completamente o foco: esse sistema que você acabou de projetar para escalar, ele é seguro?

Prática de contratos de API, e depois, segurança